

How SVD Solved the “Who Rated What” Prediction (KDD Cup 2007, Task 1)

Francisco Gutiérrez

What Problem Are We Solving?

KDD Cup 2007’s Task 1 asks: **given a list of 100,000 (user, movie) pairs from the Netflix Prize dataset, predict the probability that the user rated that movie in 2006** — not *what rating* they gave, just *whether a rating exists*. This is a binary existence-prediction problem hiding inside a matrix with hundreds of thousands of users and thousands of movies, almost entirely empty (most users haven’t rated most movies). The challenge is: how do you find structure in such a huge, sparse matrix well enough to guess which empty cells are more “likely” to have been filled in? SVD is one of four techniques the authors combine to answer this, and it turns out to be the second most valuable piece of the final model.

The Data

The input is a **user** $\times$ **movie matrix** W : $w_{ij} = 1$ if user i rated movie j , else 0.

Analogy: Topics/Themes $\leftrightarrow$ Movies

To understand why SVD helps here, it’s useful to borrow from **Latent Semantic Analysis (LSA)** in text mining. In LSA, you start with a *documents* $\times$ *terms* matrix and use SVD to discover hidden **topics** — clusters of terms that tend to co-occur across documents. A document is then described not by its raw words, but by *how much of each topic it contains*.

The same structure maps directly onto our problem:

Text Mining	Movie Rating Problem
Documents	Users
Terms	Movies
Hidden Topics	Hidden “Themes” (taste patterns)
“Document contains topic X ”	“User has taste for theme X ”
“Term belongs to topic X ”	“Movie belongs to theme X ”

Here’s the connection spelled out: just as certain **terms** cluster into a **topic** because they tend to appear in the same documents (e.g., “goal,” “striker,” “referee” $\rightarrow$ topic “soccer”), certain **movies** cluster into a **theme** because they tend to be rated by the same users (e.g., a cluster of superhero movies $\rightarrow$ theme “action blockbusters”). And just as a **document** is really a *mixture* of topics rather than belonging to just one, a **user** is a mixture of themes — someone might be 70% “action fan” and 30% “arthouse fan,” and their ratings reflect that blend.

Crucially, in LSA nobody manually labels the topics — SVD *infers* them purely from co-occurrence patterns in the data. Likewise here, nobody tells the algorithm what the themes are; SVD infers latent taste clusters purely from which movies tend to get rated by the same people.

How SVD Actually Works Here

Given this analogy, SVD factors the matrix W into three pieces:

$$W = U^T \Sigma V \tag{1}$$

- U (users $\times$ themes): each row is a user’s “theme mixture” — how strongly that user leans toward each latent theme.
- V (movies $\times$ themes): each row is a movie’s “theme profile” — how strongly that movie belongs to each latent theme.
- Σ (diagonal): how much overall “weight” or importance each theme carries across the whole dataset.

A predicted score for a specific (user, movie) pair is the **dot product of that user’s theme vector and that movie’s theme vector**, scaled by Σ — mathematically identical to computing how much a document’s topic mixture overlaps with a term’s topic profile in LSA. A high dot product means “this user’s inferred taste strongly overlaps with this movie’s latent theme,” which stands in as a proxy for “this user probably rated this movie.”

Truncation is the key trick. Using the *full* decomposition would just reconstruct W perfectly — i.e., memorize the sparse 0/1 matrix, which is useless for prediction. Instead, the authors keep only the **top $k = 10$ themes** (the 10 largest values in Σ and their corresponding vectors in U, V), discarding the rest as noise. This is justified by the **Eckart–Young theorem**, which guarantees that keeping the top k singular values/vectors gives the best possible rank- k approximation of W :

$$\|W - U_k^T \Sigma_k V_k\|_F^2 = \sum_{ij} \left(w_{ij} - \sum_k \sigma_k u_{ki} v_{kj} \right)^2 \tag{2}$$

By compressing everything into just 10 numbers per user and per movie, the model is forced to generalize rather than overfit to the sparse observed data.

Biased Sampling

The 100,000 training pairs weren’t sampled uniformly at random — the sampling procedure itself was biased toward certain users/movies (proportional to known marginals). If ignored, this bias would distort which themes SVD discovers. So instead of a plain SVD, the authors minimize a **reweighted** version of the error: each entry w_{ij} is scaled by $\sqrt{p_{ij}}$ (the square root of its sampling probability) before running SVD, correcting for the sampling bias.

Validation

To confirm the themes actually mean something, the authors plotted the reconstructed scores for pairs that *were* rated vs. pairs that *weren’t* (Figure 1 in the paper). The two distributions were

visibly separated — rated pairs skew toward higher scores — confirming the 10 latent themes genuinely capture “does this user’s taste match this movie’s theme.”

Role in the Final Model

SVD was combined with three other predictors (naive user-movie independence baseline, item-item correlation, association rules) via linear regression:

$$0.5533 \cdot p_{um} + 0.029 \cdot \text{correlation} + 0.1987 \cdot \text{SVD} - 0.0121 \cdot \text{assoc_rules} - 0.0042 \quad (3)$$

SVD received the **second-highest weight (0.1987)**, right behind the naive baseline — showing that the latent-theme structure captured real predictive signal beyond simple popularity counts. The combined system reached **RMSE = 0.256**, winning the competition.